Лабораторная работа №8

Программирование цикла. Обработка аргументов командной строки.

Пильщиков Никита Максимович

Содержание

1 Цель работы 5

2 Задание 6

3 Выполнение лабораторной работы 7

4 Выводы 21

Список иллюстраций

3.1 Создание файла 7
3.2 Редактирование программы 8
3.3 Название рисунка 9
3.4 Изменений текста программы 9
3.5 Создание исполняемого файла 10
3.6 Процесс выполнения программы 11
3.7 Изменений текста программы 13
3.8 Создание и запуск исполняемого файла 14
3.9 Внесение текста в программу 14
3.10 Название рисунка 15
3.11 Редактирование текста файла 15
3.12 Запуск исполняемого файла и получение резудльтата 16
3.13 Изменение текста программы 17
3.14 Запуск изменённого исполняемого файла 18
3.15 Создание программы 19
3.16 Запуск изменённого исполняемого файла программы 20

Список таблиц

1 Цель работы

Приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки.

2 Задание

Разобраться в организации стека. Научиться добавлять в стек, извлекать из стека.

3 Выполнение лабораторной работы

Зайдём в каталог lab08, созданный ещё при выполнении лабораторной №2, и создадим в нём файл lab8-1.asm с помощью команды touch. Далее отредактируем его в текстовом редакторе mcedit, подставив текст из листинга 8.1 (рис. 3.1).

```
[nmpiljthikov@nmpiljthikov lab08]$ touch lab8-1.asm
[nmpiljthikov@nmpiljthikov lab08]$ mcedit lab8-1.asm
```

Рис. 3.1: Создание файла

Редактируем текст программ с помощью листинга 8.1 (рис. 3.2).

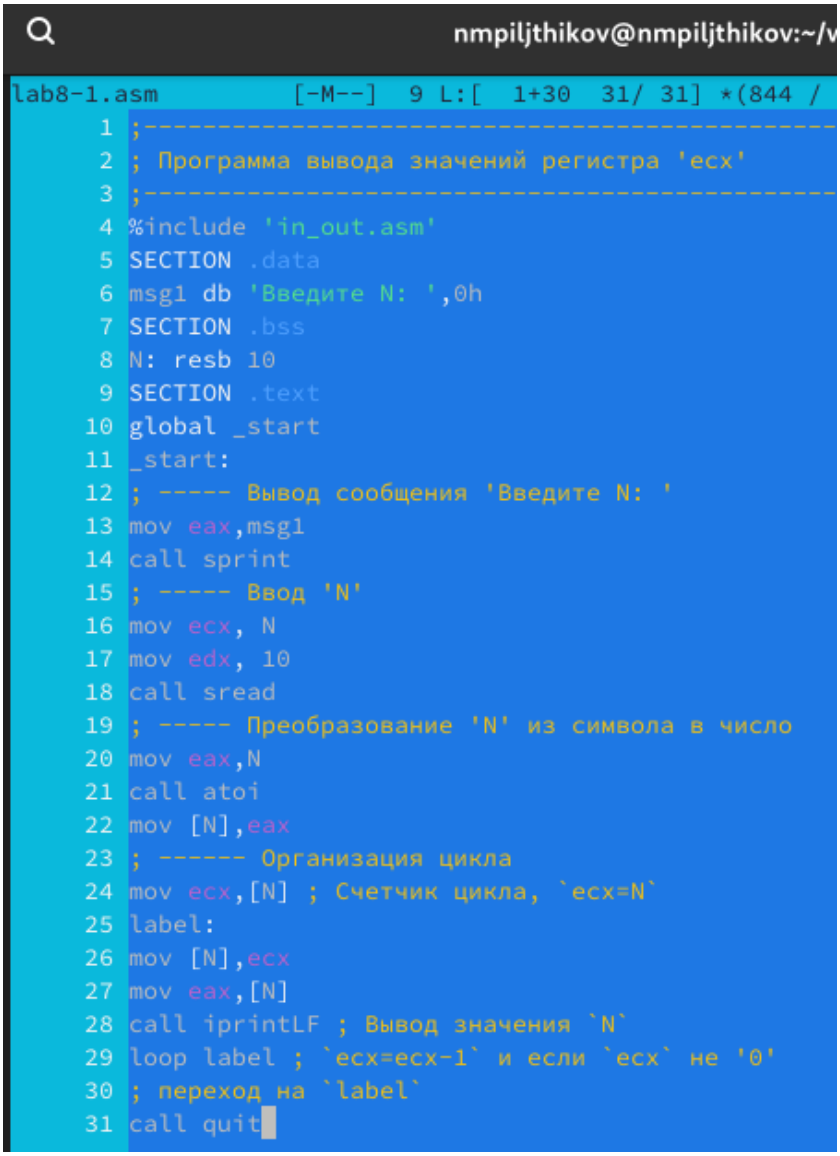

Рис. 3.2: Редактирование программы

Создаём и запускаем исполняемый файл.Проверяем результат (рис. 3.3).

```
[nmpiljthikov@nmpiljthikov lab08]$ nasm -f elf lab8-1.asm
[nmpiljthikov@nmpiljthikov lab08]$ ld -m elf_i386 -o lab8-1 lab8-1.o
[nmpiljthikov@nmpiljthikov lab08]$ ./lab8-1
Введите N: 12
12
11
10
9
8
7
6
5
4
3
2
1
```

Рис. 3.3: Название рисунка

Редактирую текст программы добавив изменение значение регистра ecx в цикле (рис. 3.4).

```
sub ecx,1 ; `ecx=ecx-1`
mov [N],ecx
mov eax,[N]
call iprintLF ; Вывод значения `N`
loop label ; `ecx=ecx-1` и если `ecx` не '0'
; переход на `label`
call quit
```

Рис. 3.4: Изменений текста программы

Создаю и запускайю исполняемый файл (рис. 3.5).

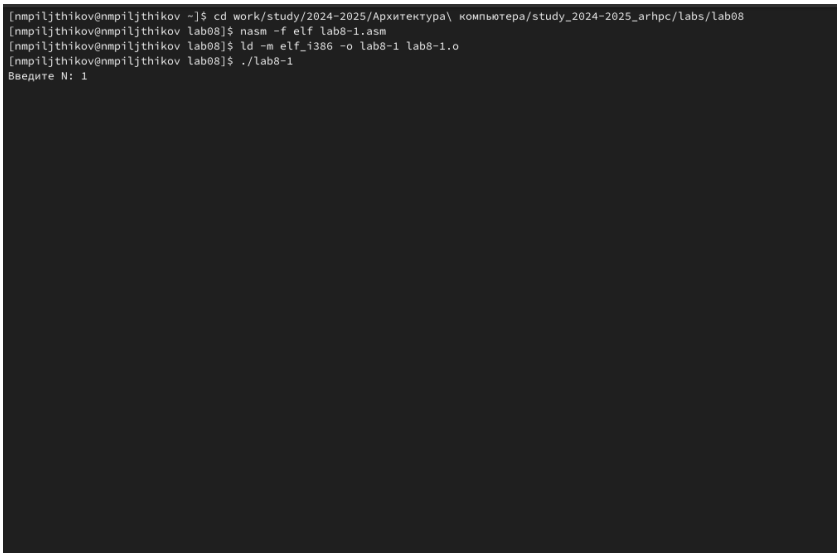

Рис. 3.5: Создание исполняемого файла

Запустив программу, та стала выдавать бесконечный цикл (рис. 3.6).

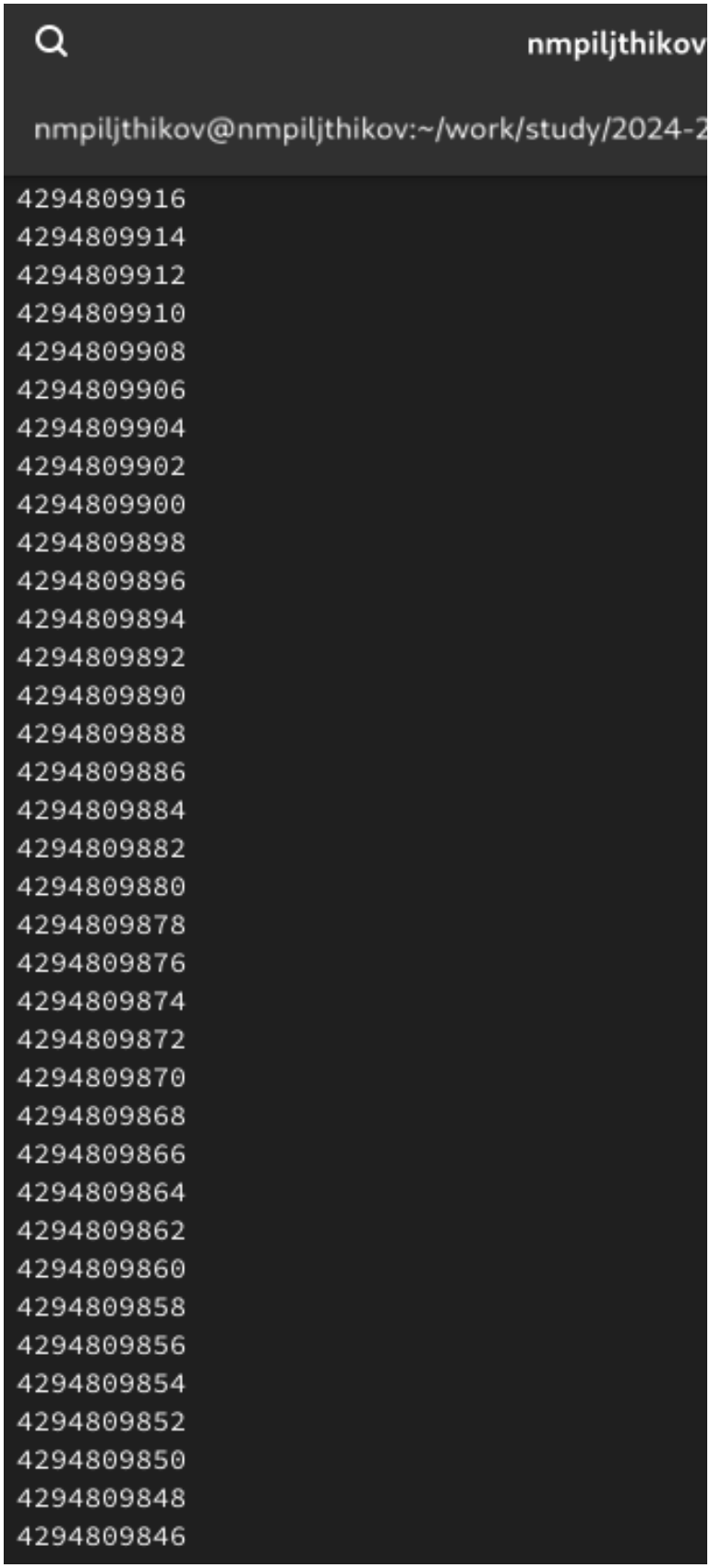

Рис. 3.6: Процесс выполнения программы

Вношу изменения в текст программы, добавив команды push и pop для сохранения значения счетчика цикла loop (рис. 3.7).

```
nmpiljthikov@
3-1.asm          [----]  7 L:[  1+30  3
;--------------------------------
; Программа вывода значений регис
;--------------------------------
%include 'in_out.asm'
SECTION .data
msg1 db 'Введите N: ',0h
SECTION .bss
N: resb 10
SECTION .text
global _start
_start:
; ----- Вывод сообщения 'Введите
mov eax,msg1
call sprint
; ----- Ввод 'N'
mov ecx, N
mov edx, 10
call sread
; ----- Преобразование 'N' из сим
mov eax,N
call atoi
mov [N],eax
; ------ Организация цикла
mov ecx,[N] ; Счетчик цикла, `ecx
label:
push ecx ; добавление значения ec
sub ecx,1
mov [N],ecx
mov eax,[N]
call iprintLF
pop ecx
loop label.
call quit
```

Рис. 3.7: Изменений текста программы

Создам и запущу исполняемый файл (рис. 3.8).

```
[nmpiljthikov@nmpiljthikov lab08]$ nasm -f elf lab8-1.asm
[nmpiljthikov@nmpiljthikov lab08]$ ld -m elf_i386 -o lab8-1 lab8-1.o
[nmpiljthikov@nmpiljthikov lab08]$ ./lab8-1
Введите N: 12
11
10
9
8
7
6
5
4
3
2
1
0
```

Рис. 3.8: Создание и запуск исполняемого файла

Теперь число проходов цикла соответствует значению, введённому с клавиатуры.Создам файл lab8-2.asm, также, как и создавался lab8-1.asm, внесём в него текст из листинга 8.2 (рис. 3.9).

```
nmpiljthikov@nmpiljthikov:~/work
lab8-2.asm        [-M--]  9 L:[  1+19  20/ 20] *(943 / 943
%include 'in_out.asm'
SECTION .text
global _start
_start:
pop ecx ; Извлекаем из стека в `ecx` количество
; аргументов (первое значение в стеке)
pop edx ; Извлекаем из стека в `edx` имя программы
; (второе значение в стеке)
sub ecx, 1 ; Уменьшаем `ecx` на 1 (количество
; аргументов без названия программы)
next:
cmp ecx, 0 ; проверяем, есть ли еще аргументы
jz _end ; если аргументов нет выходим из цикла
; (переход на метку `_end`)
pop eax ; иначе извлекаем аргумент из стека
call sprintLF ; вызываем функцию печати
loop next ; переход к обработке следующего
; аргумента (переход на метку `next`)
_end:
call quit
```

Рис. 3.9: Внесение текста в программу

Создание исполняемого файла, и его последующий запуск с аргументом 1, аргументом 2, аргументом 3 (рис. 3.10).

```
[nmpiljthikov@nmpiljthikov lab08]$ nasm -f elf lab8-2.asm
[nmpiljthikov@nmpiljthikov lab08]$ ld -m elf_i386 -o lab8-2 lab8-2.o
[nmpiljthikov@nmpiljthikov lab08]$ ./lab8-2 аргумент1 аргумент 2 'аргумент 3'
аргумент1
аргумент
2
аргумент 3
```

Рис. 3.10: Название рисунка

Команда обработала 4 аргумента, так как “аргумент” и “2” считаются разными аргументами, потому что между ними стоит пробел. Создаю файл lab8-3.asm для следующей программы и сразу вношу него текст программы (рис. 3.11).

```
nmpiljthikov@nmpiljthikov:~/work/st
lab8-3.asm        [----] 32 L:[  1+28  29/ 29] *(1428/1428b)
%include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
pop ecx ; Извлекаем из стека в `ecx` количество
; аргументов (первое значение в стеке)
pop edx ; Извлекаем из стека в `edx` имя программы
; (второе значение в стеке)
sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
; аргументов без названия программы)
mov esi, 0 ; Используем `esi` для хранения
; промежуточных сумм
next:
cmp ecx,0h ; проверяем, есть ли еще аргументы
jz _end ; если аргументов нет выходим из цикла
; (переход на метку `_end`)
pop eax ; иначе извлекаем следующий аргумент из стека
call atoi ; преобразуем символ в число
add esi,eax ; добавляем к промежуточной сумме
; след. аргумент `esi=esi+eax`
loop next ; переход к обработке следующего аргумента
_end:
mov eax, msg ; вывод сообщения "Результат: "
call sprint
mov eax, esi ; записываем сумму в регистр `eax`
call iprintLF ; печать результата
call quit ; завершение программы
```

Рис. 3.11: Редактирование текста файла

Создаю и запускаю исполняемый файл с аргументами: 13 14 8 11 6.Результат работы программы. Сумма аргументов командной строки 13+14+8+11+6=52. Это совпадает с результатом программы, следовательно она работает правильно (рис. 3.12).

```
[nmpiljthikov@nmpiljthikov lab08]$ nasm -f elf lab8-3.asm
[nmpiljthikov@nmpiljthikov lab08]$ ld -m elf_i386 -o lab8-3 lab8-3.o
[nmpiljthikov@nmpiljthikov lab08]$ ./lab8-3 13 14 8 11 6
Результат: 52
```

Рис. 3.12: Запуск исполняемого файла и получение резудльтата

Изменяю текст программы lab8-3.asm для вычисления произведения аргументов командной строки (рис. 3.13).

```
nmpiljthikov@nmpiljthikov:~/work/
sm          [-M--]  9 L:[  1+16  17/ 32] *(637 /1230b
%include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
pop ecx ; Извлекаем из стека в `ecx` количество
; аргументов (первое значение в стеке)
pop edx ; Извлекаем из стека в `edx` имя программы
; (второе значение в стеке)
sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
; аргументов без названия программы)
mov esi, 1 ; Используем `esi` для хранения
; промежуточных сумм
mov eax, 1
next:
cmp ecx,0 ; проверяем, есть ли еще аргументы
jz _end ; если аргументов нет выходим из цикла
; (переход на метку `_end`)
pop eax
call atoi
mov ebx,eax
mov eax,esi
imul ebx
mov esi,eax
loop next ; переход к обработке следующего аргумента
_end:
mov eax, msg ; вывод сообщения "Результат: "
call sprint
mov eax, esi ; записываем сумму в регистр `eax`
call iprintLF ; печать результата
call quit ; завершение программы
```

Рис. 3.13: Изменение текста программы

Запустим созданный мною исполняемый файл и убедимся в том, что всё работает правильно (рис. 3.14).

```
[nmpiljthikov@nmpiljthikov lab08]$ nasm -f elf lab8-3.asm
[nmpiljthikov@nmpiljthikov lab08]$ ld -m elf_i386 -o lab8-3 lab8-3.o
[nmpiljthikov@nmpiljthikov lab08]$ ./lab8-3 4 2
Результат: 8
[nmpiljthikov@nmpiljthikov lab08]$ ./lab8-3 5 5 4
Результат: 100
```

Рис. 3.14: Запуск изменённого исполняемого файла

Программа работает верно. При аргументах 4 и 2 программа выдаёт ответ умножения 8, а при подставлении 5 5 4 - ответ 100, следовательно умножение выполнено правильно.

#Самостоятельная работа

Создаю файл lab8-4.asm и ввожу текст программы для вычисления суммы функций вида f(x)=4x-3. Выполнение самостоятельного задание ведётся в соотвествии с вариантом № 6, полученным из лабороторной № 7 (рис. 3.15).

```
nmpiljthikov@
asm          [----] 10 L:[  1+36  37
%include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
msg1 db "Функция: f(x)=4x-3",0
SECTION .text
global _start
_start:
mov eax, msg1
call sprintLF

pop ecx.
pop edx.
sub ecx,1.
mov esi,0.

next:
cmp ecx,0h
jz _end

pop eax

call atoi

mov ebx, 4
mul ebx....
sub eax, 3.
add esi, eax
mov eax, esi

loop next.

_end:
mov eax, msg.
call sprint
mov eax, esi
call iprintLF
call quit.
```

Рис. 3.15: Создание программы

Далее запустим исполняемый файл программы и подставим разные аргументы (рис. 3.16).

```
[nmpiljthikov@nmpiljthikov lab08]$ ld -m elf_i386 -o lab8-4 lab8-4.o
[nmpiljthikov@nmpiljthikov lab08]$ ./lab8-4 1 2 3 4
Функция: f(x)=4x-3
Результат: 28
[nmpiljthikov@nmpiljthikov lab08]$ mcedit lab8-4.asm
```

Рис. 3.16: Запуск изменённого исполняемого файла программы

Ответ получен верный, программа работает правильно.

4 Выводы

Я приобрёл навыки написания программ с использованием циклов и обработкой аргументов командной строки, а также разобрался в организации стека. Научился добавлять в стек, извлекать из стека.